

Sổ tay vận hành

Triển khai, vận hành hằng ngày và xử lý sự cố

Mã tài liệu	OBE-RUN-001-VI
Phiên bản	1.1
Ngày	2026-10-10
Trạng thái	Đã duyệt cho môi trường dev
Người phụ trách	khaipd18 (DevOps / Cloud)
Đối tượng đọc	Kỹ sư triển khai, vận hành hoặc hỗ trợ nền tảng
Phân loại	Nội bộ
Repository	https://github.com/khaipd18/online-boutique-gitops-pipeline

Kiểm soát tài liệu

Lịch sử thay đổi

Phiên bản	Ngày	Thay đổi	Người thực hiện
1.0	2026-10-09	Phát hành lần đầu: truy cập, bootstrap, quy trình thường ngày, monitoring, xử lý sự cố, gỡ môi trường.	khaipd18
1.1	2026-10-10	Chạy thật trên tài khoản mới: đổi tên bucket state, Terraform quản lý log group của control plane (74 resource), đã kiểm chứng apply và destroy.	khaipd18

Tài liệu liên quan

- OBE-TDD-001-VI Tài liệu thiết kế kỹ thuật (docs/manuals/technical-design.vi.pdf)
- Bản tiếng Anh của sổ tay này: docs/manuals/operations-runbook.en.pdf (OBE-RUN-001)
- README.vi.md: tóm tắt dự án và hướng dẫn chạy thử với kind

Mục lục

1	Về sổ tay này	3
1.1	Mục đích và cách dùng	3
1.2	Thông tin môi trường	3
1.3	Mức độ nghiêm trọng	3
2	Truy cập và công cụ	4
2.1	Công cụ trên máy làm việc	4
2.2	Kết nối tới AWS và cluster	4
2.3	Mở các giao diện quản trị	4
3	Quy trình làm một lần	5
3.1	RB-01 Dựng môi trường mới	5
3.2	RB-02 Gỡ môi trường	6
3.3	RB-03 Chuyển sang tài khoản AWS khác	6
4	Quy trình thường ngày	7
4.1	RB-04 Ra bản mới cho một service	7
4.2	RB-05 Rollback một service	7
4.3	RB-06 Đổi cấu hình service	7
4.4	RB-07 Thêm service mới	7
4.5	RB-08 Thay đổi hạ tầng	8
4.6	RB-09 Scale	8
4.7	RB-10 Nâng phiên bản Kubernetes và add-on	8
4.8	RB-11 Chấp nhận hoặc sửa một finding Checkov	9
4.9	RB-12 Xử lý lỗi hỏng trong image	9
4.10	RB-13 Kiểm soát chi phí	9
5	Monitoring	10
5.1	Kiểm tra hằng ngày	10
5.2	Cần theo dõi gì trên Grafana	10
5.3	Log	10
6	Xử lý sự cố	11
6.1	IR-01 CI fail ở bước "Configure AWS credentials"	11
6.2	IR-02 Terraform state bị khóa không nhả	11
6.3	IR-03 Terraform Apply không chạy	11
6.4	IR-04 GitOps write-back bị từ chối	11
6.5	IR-05 Application của Argo CD bị OutOfSync hoặc Degraded	12
6.6	IR-06 Pod bị ImagePullBackOff	12
6.7	IR-07 Pod bị CrashLoopBackOff hoặc OOMKilled	12
6.8	IR-08 Pod bị Pod Security Admission từ chối	12
6.9	IR-09 Lời gọi giữa các service bị timeout	13
6.10	IR-10 Không vào được shop từ internet	13
6.11	IR-11 Pod bị Pending hoặc node NotReady	13
6.12	IR-12 HPA hiện metrics <unknown>	13
6.13	IR-13 Không drain được node	14
6.14	IR-14 Giỏ hàng bị trống	14
7	Khôi phục sau thảm họa	15
	Phụ lục A. Service và port	16
	Phụ lục B. Lệnh thường dùng	17
	Phụ lục C. Phiên bản	18

1 Về sổ tay này

1.1 Mục đích và cách dùng

Sổ tay này hướng dẫn kỹ sư triển khai và vận hành nền tảng Online Boutique: các quy trình làm một lần (bootstrap, gỡ môi trường, chuyển tài khoản), các thay đổi thường ngày (ra bản mới, rollback, đổi cấu hình, đổi hạ tầng) và cách xử lý khi có sự cố. Thiết kế đằng sau được mô tả trong Tài liệu thiết kế kỹ thuật (OBE-TDD-001-VI).

Mỗi quy trình ghi điều kiện cần, các bước đánh số và bước kiểm tra kết quả. Làm theo đúng thứ tự. Các lệnh giả định chạy bằng Bash tại thư mục gốc của repository.

QUAN TRỌNG

Git là nguồn sự thật duy nhất. Argo CD đưa mọi thay đổi tay bằng kubectl trong namespace dev-eks về lại như cũ (self-heal). Ngoài các lệnh chẩn đoán trong sổ tay này, hãy thay đổi cluster qua pull request.

1.2 Thông tin môi trường

Mục	Giá trị
Region	ap-southeast-1
EKS cluster	khaipd18-eks-cluster (Kubernetes 1.35)
Namespace ứng dụng	dev-eks (release <service>-dev)
Namespace khác	argocd (Argo CD), monitoring (kube-prometheus-stack)
Điểm vào public	Service frontend-external-dev (Classic Load Balancer, port 80)
Terraform state	S3 khaipd18-obe-tf-state-445817183958, key dev/terraform.tfstate bảng khóa khaipd18-devops-project-terraform-state-lock
Repository	https://github.com/khaipd18/online-boutique-gitops-pipeline
Kiểm soát trên GitHub	Ruleset protect-main; environment production (người duyệt khaipd18)

1.3 Mức độ nghiêm trọng

Mức	Định nghĩa	Ví dụ	Phản hồi
SEV1	Shop không truy cập được hoặc checkout hỏng với mọi người dùng	frontend sập, mất load balancer, mọi pod đều lỗi	Xử lý ngay; ưu tiên sửa hoặc rollback, điều tra sau
SEV2	Một tính năng bị suy giảm, hoặc luồng triển khai bị chặn	Một service crash liên tục, CI không push được image, Argo CD bị kẹt	Trong ngày làm việc
SEV3	Không ảnh hưởng người dùng	Finding scan mức warning, một pod khởi động lại, tài liệu lệch với thực tế	Đưa vào kế hoạch thường ngày

Nền tảng có một người phụ trách duy nhất (khaipd18). Mọi sự cố SEV1/SEV2 được ghi lại thành GitHub issue, gồm diễn biến, nguyên nhân và việc cần làm tiếp.

2 Truy cập và công cụ

2.1 Công cụ trên máy làm việc

Công cụ	Dùng để
AWS CLI v2	Credential, kubeconfig, backend state, kiểm tra trên AWS
Terraform 1.14.8	Hạ tầng (chỉ cho lần apply đầu và trường hợp khẩn cấp; bình thường do CI chạy)
kubectl, Helm	Kiểm tra cluster; cài Argo CD
GitHub CLI (gh)	Xem run, log, duyệt deployment, cài đặt repository
Typst	Build lại các tài liệu PDF này (docs/manuals/build.sh)

2.2 Kết nối tới AWS và cluster

1. Kiểm tra danh tính AWS và tài khoản đang dùng:

```
aws sts get-caller-identity
```

2. Ghi cấu hình kubeconfig:

```
aws eks update-kubeconfig --region ap-southeast-1 --name khaipd18-eks-cluster
kubectl get nodes
```

GHI CHÚ

Cluster không đặt access_config nên áp dụng mặc định của API: ban đầu chỉ IAM principal đã tạo cluster có quyền admin [AWS]. Kiểm tra chế độ bằng `aws eks describe-cluster --name khaipd18-eks-cluster --query cluster.accessConfig`. Muốn cấp quyền cho kỹ sư khác, chuyển cluster sang API_AND_CONFIG_MAP rồi tạo access entry kèm access policy [AWS] nên làm bằng Terraform để không bị mất ở lần apply sau.

2.3 Mở các giao diện quản trị

Mọi giao diện đều truy cập qua kubectl port-forward; không giao diện nào mở ra internet.

```
# Argo CD: https://localhost:8080, user admin
kubectl -n argocd port-forward svc/argocd-server 8080:443
kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath='{.data.password}' | base64 -d; echo

# Grafana: http://localhost:3000, user admin
kubectl -n monitoring port-forward svc/kube-prometheus-stack-grafana 3000:80
kubectl -n monitoring get secret kube-prometheus-stack-grafana \
  -o jsonpath='{.data.admin-password}' | base64 -d; echo

# Prometheus: http://localhost:9090
kubectl -n monitoring port-forward svc/kube-prometheus-stack-prometheus 9090:9090

# Địa chỉ shop
kubectl -n dev-eks get svc frontend-external-dev \
  -o jsonpath='{.status.loadBalancer.ingress[0].hostname}'; echo
```

Đổi mật khẩu admin của Argo CD sau lần đăng nhập đầu tiên và xóa argocd-initial-admin-secret.

3 Quy trình làm một lần

3.1 RB-01 Dựng môi trường mới

Khi nào: lần triển khai đầu tiên trên một tài khoản AWS, hoặc dựng lại sau khi đã gỡ. **Thời gian:** khoảng 45 phút (tạo EKS mất 15–20 phút). **Chi phí:** bắt đầu tính theo giờ từ bước 4 (xem RB-13).

1. **Đặt tài khoản đích trên GitHub.** Settings → Secrets and variables → Actions → Variables → AWS_ACCOUNT_ID. Hoặc:

```
gh variable set AWS_ACCOUNT_ID --body <account-id>
```

2. **Kiểm tra trùng tên** nếu tài khoản dùng chung: mỗi tài khoản chỉ có một OIDC provider cho token.actions.githubusercontent.com, và tên IAM role, policy là duy nhất trong tài khoản (github-actions-*, GitHubActions-*).
3. **Tạo backend lưu state** (một lần cho mỗi tài khoản). Tên bucket là duy nhất toàn cầu; nếu đổi tên thì sửa cả terraform/backend.tf và các biến tf_state_bucket / tf_state_lock_table.

```
aws s3api create-bucket --bucket <state-bucket> --region ap-southeast-1 \
  --create-bucket-configuration LocationConstraint=ap-southeast-1
aws s3api put-bucket-versioning --bucket <state-bucket> \
  --versioning-configuration Status=Enabled
aws dynamodb create-table --table-name <lock-table> --region ap-southeast-1 \
  --attribute-definitions AttributeName=LockID,AttributeType=S \
  --key-schema AttributeName=LockID,KeyType=HASH --billing-mode PAY_PER_REQUEST
```

4. **Apply lần đầu từ máy local.** Lúc này các role cho GitHub chưa tồn tại nên CI chưa làm được. Danh tính dùng ở bước này sẽ trở thành admin của cluster.

```
cd terraform
aws sts get-caller-identity          # phải đúng tài khoản đích
terraform init
terraform plan -out tfplan          # 74 resource sẽ được tạo
terraform apply tfplan
```

5. **Cài Argo CD và các ứng dụng:**

```
aws eks update-kubeconfig --region ap-southeast-1 --name khaipd18-eks-cluster
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/
stable/manifests/install.yaml
kubectl apply -f gitops/argocd/namespaces.yaml
kubectl apply -f gitops/argocd/applicationset.yaml
kubectl apply -f gitops/argocd/monitoring.yaml --server-side
```

6. **Build image.** Chạy 5 workflow CI một lần (Actions → CI for ... Services → Run workflow), hoặc:

```
for w in go dotnet java node py; do gh workflow run $w-services-ci.yaml --ref main;
done
```

CI push image và ghi tag mới vào gitops/dev-eks/.

CẢNH BÁO

Vấn đề đã biết: ruleset protect-main từ chối lần push tag image mới của bot CI (lỗi GitHub GH013), nên bước 6 fail ở "Update GitOps and Push Back". Trong lúc chờ bản sửa ở roadmap, hãy tắt ruleset trong lúc bootstrap và bật lại ngay sau đó:

```
gh api -X PUT repos/khaipd18/online-boutique-gitops-pipeline/rulesets/24705847 -f
enforcement=disabled
# ... chạy bước 6 và chờ mọi run kết thúc ...
gh api -X PUT repos/khaipd18/online-boutique-gitops-pipeline/rulesets/24705847 -f
enforcement=active
```

Kiểm tra:

```
kubectl -n argocd get applications           # tất cả Synced / Healthy
kubectl -n dev-eks get pods                 # tất cả Running, frontend có 2 replica
kubectl -n dev-eks get hpa,pdb              # HPA của frontend đọc được giá trị CPU
```

Mở địa chỉ shop và đặt thử một đơn; trang xác nhận đơn hàng phải hiện ra.

3.2 RB-02 Gỡ môi trường

Khi nào: không còn cần môi trường. EKS không có free tier; control plane, node EC2, NAT Gateway và interface endpoint đều tính tiền theo giờ [\[AWS\]](#).

1. Xóa các ứng dụng và load balancer trước. Load balancer do Kubernetes tạo nên không nằm trong Terraform state; nếu còn sót, terraform destroy sẽ fail ở bước xóa VPC.

```
kubectl delete -f gitops/argocd/applicationset.yaml
kubectl -n dev-eks delete svc frontend-external-dev --ignore-not-found
```

2. Gỡ hạ tầng (từ máy local, bằng credential admin):

```
cd terraform && terraform destroy
```

3. Tùy chọn: xóa bucket state và bảng khóa nếu không dùng tài khoản này nữa.
4. Tùy chọn: xóa biến trên GitHub để CI lại bỏ qua các bước AWS: `gh variable delete AWS_ACCOUNT_ID`.

Kiểm tra: `aws eks list-clusters --region ap-southeast-1` không còn cluster, và EC2 console không còn load balancer hay NAT Gateway nào trong region.

3.3 RB-03 Chuyển sang tài khoản AWS khác

1. Chạy RB-02 ở tài khoản cũ (hoặc chấp nhận để nó tiếp tục chạy).
2. Chạy RB-01 ở tài khoản mới. Không cần sửa code: Terraform lấy tài khoản từ credentials, còn CI lấy từ `AWS_ACCOUNT_ID`.
3. Địa chỉ ECR trong `gitops/dev-eks/values-*.yaml` vẫn trở về tài khoản cũ cho tới khi CI ghi đè ở bước 6 của RB-01.

4 Quy trình thường ngày

4.1 RB-04 Ra bản mới cho một service

1. Merge thay đổi trong `src/<service>/` vào `main`.
2. Workflow của ngôn ngữ đó lint, test, build, quét (Trivy) rồi push image tag bằng commit SHA, sau đó commit tag mới vào `gitops/dev-eks/values-<service>.yaml` kèm `[skip ci]`.
3. Argo CD phát hiện commit (mặc định kiểm tra mỗi 3 phút) và rollout image mới.

Kiểm tra:

```
gh run list --limit 5
kubectl -n argocd get application <service>-dev
kubectl -n dev-eks rollout status deploy/<service>-dev
kubectl -n dev-eks get deploy <service>-dev -o jsonpath='{..image}'; echo
```

4.2 RB-05 Rollback một service

Mỗi tag image là một commit SHA và tag không đổi được, nên rollback nghĩa là cho file values trở lại tag trước đó.

1. Tìm commit đã đổi tag:

```
git log --oneline -- gitops/dev-eks/values-<service>.yaml
```

2. Revert commit đó qua pull request (hoặc push thẳng bằng quyền admin khi khẩn cấp):

```
git revert <commit-of-bad-tag>
```

3. Argo CD đưa service về image trước đó. Theo dõi bằng `kubectl -n dev-eks rollout status deploy/<service>-dev`.

CẢNH BÁO

Không dùng `kubectl set image` hay `kubectl rollout undo`: Argo CD self-heal sẽ khôi phục tag trong Git chỉ sau vài phút.

4.3 RB-06 Đổi cấu hình service

Biến môi trường, resource, probe, số replica, autoscaling và danh sách caller của NetworkPolicy đều nằm trong `gitops/dev-eks/values-<service>.yaml`.

1. Sửa file values trên một nhánh và render thử ở máy local:

```
helm template <service>-dev helm-charts -f gitops/dev-eks/values-<service>.yaml -n dev-eks
```

2. Mở pull request; workflow Security Scan chạy Checkov trên manifest đã render.
3. Merge; Argo CD áp dụng thay đổi.

MẸO

Khi một service bắt đầu gọi sang service khác, thêm tên release của bên gọi vào `networkPolicy.allowFrom` của bên được gọi, nếu không lời gọi sẽ bị timeout (xem IR-09).

4.4 RB-07 Thêm service mới

1. Thêm `gitops/dev-eks/values-<name>.yaml` (chép từ một service tương tự; giữ nguyên thiết lập bảo mật mặc định của chart).

2. Thêm - name: <name> vào list generator trong gitops/argocd/applicationset.yaml.
3. Thêm service vào allowFrom của những service mà nó gọi, và liệt kê các caller của nó trong allowFrom của chính nó.
4. Nếu có image mới: thêm tên repository ECR vào repository_names trong terraform/variables.tf, và thêm service vào workflow CI tương ứng (paths filter và danh sách khi chạy tay).
5. Apply thay đổi của ApplicationSet một lần: `kubectl apply -f gitops/argocd/applicationset.yaml` (bản thân ApplicationSet không do Argo CD quản lý).

4.5 RB-08 Thay đổi hạ tầng

1. Sửa terraform/ trên một nhánh và mở pull request.
2. **Checkov Scan** phải pass. **Terraform Plan** chạy bằng role read-only; đọc plan trong log của job.
3. Nhờ review pull request, rồi merge.
4. Job **Terraform Apply** chờ duyệt. Duyệt ở Actions → run tương ứng → Review deployments → production → Approve and deploy, hoặc:

```
gh run list --workflow terraform.yaml --limit 1
gh run view <run-id> # hiện "waiting for review"
```

5. Xem log apply, rồi kiểm tra resource trên AWS.

QUAN TRỌNG

Chỉ duyệt khi đã đọc plan. Sau khi được duyệt, job apply chạy `terraform apply -auto-approve` với quyền AdministratorAccess.

4.6 RB-09 Scale

- **frontend:** HPA giữ 2–4 replica ở mức 70 % CPU. Điều chỉnh khoảng này trong values-frontend.yaml (autoscaling.minReplicas / maxReplicas).
- **Service khác:** đặt replicaCount hoặc bật autoscaling.enabled trong file values. PodDisruptionBudget tự được thêm từ 2 replica. Giữ redis-cart ở 1 (dữ liệu nằm trong emptyDir).
- **Node:** đổi eks_node_group_scaling_config (min/desired/max, hiện 1/2/3) theo RB-08. Chưa có cluster autoscaler: desired_size chính là số node.

4.7 RB-10 Nâng phiên bản Kubernetes và add-on

Kubernetes 1.35 hết standard support ngày 27/03/2027 [AWS]. Nâng từng minor version một, qua pull request (RB-08):

1. Đọc release note của EKS cho phiên bản đích và kiểm tra API bị deprecated trong manifest.
2. Tăng eks_k8s_version rồi apply: bước này chỉ nâng control plane.
3. Nâng managed node group. Module Terraform không đặt version cho node group, nên node giữ phiên bản cũ cho tới khi được cập nhật, và không được mới hơn control plane [AWS]:

```
aws eks update-nodegroup-version --cluster-name khaipd18-eks-cluster \
  --nodegroup-name khaipd18-eks-cluster-node-group --kubernetes-version <version>
```

4. Tìm phiên bản add-on cho Kubernetes mới rồi tăng các biến eks_*_version:

```
aws eks describe-addon-versions --addon-name vpc-cni --kubernetes-version <version> \
  --query 'addons[0].addonVersions[0:3].
[addonVersion,compatibilities[0].defaultVersion]' --output text
```

Làm tương tự cho coredns, kube-proxy và metrics-server.

Kiểm tra: `kubectl get nodes` cho thấy mọi node đã lên phiên bản mới; mọi Application của Argo CD đều Healthy; checkout trên shop vẫn chạy.

4.8 RB-11 Chấp nhận hoặc sửa một finding Checkov

- Ưu tiên sửa Terraform. Nếu finding chấp nhận được, ghi rõ lý do (comment ngay cạnh resource).
- Tạo lại baseline trong một pull request để được review:

```
checkov --config-file .checkov.yaml -d terraform --framework terraform --create-baseline
```

4.9 RB-12 Xử lý lỗ hổng trong image

- Mở tab **Security** của GitHub → Code scanning, lọc theo tool **Trivy** và image (trivy-<service>). Mỗi run cũng ghi số lỗ hổng của từng image trong job summary.
- Finding đến từ base image và dependency trong `src/`, phần repository này không sửa. Nâng cấp chúng là việc thay đổi code của chủ ứng dụng; sau đó repository này chuyển gate sang blocking bằng cách đặt blocking: 'true' ở bước Trivy.
- SBOM (CycloneDX) của từng image được lưu thành artifact của run trong 30 ngày (sbom-<service>).

4.10 RB-13 Kiểm soát chi phí

Hạng mục tính phí	Ghi chú
EKS control plane	Tính theo giờ khi cluster còn tồn tại
Node EC2	Mặc định 2 × t3.medium on-demand
NAT Gateway	Theo giờ và theo GB xử lý [AWS]
Interface endpoint	3 endpoint × 2 AZ, theo giờ và theo GB
Classic Load Balancer	Theo giờ và theo GB
CloudWatch Logs	5 loại control plane log; lưu 365 ngày, bị xóa cùng cluster
ECR, S3, DynamoDB	Nhỏ: dung lượng lưu trữ và số request

Kiểm tra chi phí thực tế bằng AWS Cost Explorer của tài khoản. Gỡ môi trường (RB-02) khi không dùng.

5 Monitoring

5.1 Kiểm tra hằng ngày

```
kubectl -n argocd get applications          # Synced / Healthy
kubectl -n dev-eks get pods                # Running, số lần restart không tăng
kubectl -n dev-eks get hpa                 # CPU của frontend dưới mục tiêu
kubectl get nodes                           # Ready
gh run list --limit 10                     # kết quả CI gần đây
```

5.2 Cần theo dõi gì trên Grafana

kube-prometheus-stack có sẵn dashboard cho cluster, node, namespace và workload. Theo dõi:

- Số lần pod restart và container bị OOMKilled trong dev-eks (dashboard **Kubernetes / Compute Resources / Namespace (Pods)**).
- CPU và memory của từng pod so với request và limit ở Phụ lục A.
- CPU và memory của node: với 2 node t3.medium, cluster không còn nhiều dư địa.
- Số replica của frontend (hoạt động của HPA).

Hiện chưa có alert rule cho ứng dụng; Alertmanager chỉ có các rule mặc định. Alert theo SLO nằm trong roadmap.

5.3 Log

- Log của pod: `kubectl -n dev-eks logs deploy/<service>-dev --tail 100` (thêm `--previous` sau khi crash).
- Control plane log (api, audit, authenticator, controllerManager, scheduler): log group `/aws/eks/khaipd18-eks-cluster/cluster` trên CloudWatch Logs.
- Log CI: `gh run view <run-id> --log-failed`.

6 Xử lý sự cố

6.1 IR-01 CI fail ở bước “Configure AWS credentials”

Triệu chứng	Could not assume role with OIDC: Not authorized to perform sts:AssumeRoleWithWebIdentity hoặc The web identity token provided could not be validated.
Nguyên nhân thường gặp	AWS_ACCOUNT_ID trỏ tới tài khoản sai hoặc đã bỏ; role hay OIDC provider chưa tồn tại ở đó; claim sub của token không khớp trust policy (ví dụ job apply không dùng environment production).
Chẩn đoán	gh variable list; trong tài khoản: aws iam get-role --role-name github-actions-ecr-oidc-role --query Role.AssumeRolePolicyDocument; so các giá trị sub được phép với job (nhánh main, pull_request hay environment:production).
Xử lý	Sửa biến, hoặc chạy bước 4 của RB-01 để tạo role. Muốn tạm dừng hẳn các bước AWS, xóa biến đi: CI khi đó chỉ lint, test, build và scan.

6.2 IR-02 Terraform state bị khóa không nhả

Triệu chứng	Error acquiring the state lock kèm một lock ID, dù không có apply nào đang chạy.
Nguyên nhân thường gặp	Một run trước bị hủy hoặc crash khi đang giữ khóa DynamoDB.
Chẩn đoán	Chắc chắn không có job Terraform nào đang chạy: gh run list --workflow terraform.yml. Ghi lại lock ID và ai đang giữ khóa từ thông báo lỗi.
Xử lý	Từ máy local với credential admin: cd terraform && terraform force-unlock <lock-id>. Tuyệt đối không mở khóa khi có apply khác đang chạy: hai lần apply song song có thể làm hỏng state. Bucket state có versioning nên state hỏng có thể khôi phục từ phiên bản object trước đó.

6.3 IR-03 Terraform Apply không chạy

Triệu chứng	Run hiện Waiting ở job Terraform Apply.
Nguyên nhân thường gặp	Environment production cần người duyệt.
Chẩn đoán	gh run view <run-id>; job đang chờ review deployment.
Xử lý	Đọc plan trong pull request, rồi duyệt ở trang của run (Review deployments). Từ chối nếu plan khác với những gì đã review.

6.4 IR-04 GitOps write-back bị từ chối

Triệu chứng	Bước “Update GitOps and Push Back” fail với GH013: Repository rule violations found / Changes must be made through a pull request. Image đã có trên ECR nhưng cluster vẫn chạy tag cũ.
Nguyên nhân thường gặp	Ruleset protect-main bắt buộc qua pull request, và repository cá nhân không đưa được app GitHub Actions vào danh sách bypass.
Chẩn đoán	gh run view <run-id> --log-failed.
Xử lý	Trước mắt: tự cập nhật tag trong gitops/dev-eks/values-<service>.yaml (qua pull request hoặc push bằng quyền admin), lấy SHA từ run; hoặc tắt ruleset trong lúc CI chạy (cảnh báo ở RB-01). Sửa lâu dài (roadmap): deploy key trong danh sách bypass, hoặc cho bot mở pull request.

6.5 IR-05 Application của Argo CD bị OutOfSync hoặc Degraded

Triệu chứng	kubectl -n argocd get applications hiện OutOfSync, Degraded hoặc Progressing trong thời gian dài.
Nguyên nhân thường gặp	Values sai (lỗi render), pod không bao giờ ready, resource bị sửa tay, hoặc một field do controller khác quản lý.
Chẩn đoán	kubectl -n argocd describe application <service>-dev (conditions và kết quả sync); kubectl -n dev-eks get events --sort-by=.lastTimestamp tail -20; render thử ở local bằng helm template.
Xử lý	Sửa values trong Git. Nếu sync bị kẹt, refresh trên giao diện Argo CD (Refresh → Hard refresh) hoặc kubectl -n argocd annotate application <service>-dev argocd.argoproj.io/refresh=hard --overwrite.

6.6 IR-06 Pod bị ImagePullBackOff

Triệu chứng	Pod ở trạng thái ImagePullBackOff hoặc ErrImagePull.
Nguyên nhân thường gặp	Tag không có trên ECR (CI chưa push, hoặc values vẫn trỏ tới tài khoản khác); node không tới được ECR.
Chẩn đoán	kubectl -n dev-eks describe pod <pod> (phần Events ghi image và lỗi); aws ecr describe-images --repository-name <service> --image-ids imageTag=<tag>; kiểm tra account trong địa chỉ image có đúng tài khoản hiện tại.
Xử lý	Chạy workflow CI của service để build và push tag, hoặc rollback về tag đã có (RB-05). Nếu không tới được ECR, kiểm tra endpoint ecr.api/ecn.dkr và route tới S3 gateway endpoint trong route table private.

6.7 IR-07 Pod bị CrashLoopBackOff hoặc OOMKilled

Triệu chứng	Số lần restart tăng dần; kubectl get pods hiện CrashLoopBackOff; describe hiện Last State: Terminated, Reason: OOMKilled.
Nguyên nhân thường gặp	Memory limit quá thấp, biến môi trường sai, hoặc một service phụ thuộc chưa sẵn sàng lúc khởi động.
Chẩn đoán	kubectl -n dev-eks logs <pod> --previous; kubectl -n dev-eks describe pod <pod>; so mức dùng trên Grafana với limit.
Xử lý	Sửa values (RB-06) hoặc rollback (RB-05). Ví dụ đã gặp: GOMEMLIMIT: "230Mi" làm frontend crash ngay khi khởi động vì Go chỉ nhận đơn vị kiểu MiB; bản sửa là 230MiB (commit feb47a7).

6.8 IR-08 Pod bị Pod Security Admission từ chối

Triệu chứng	Event của ReplicaSet hiện Error creating: pods ... is forbidden: violates PodSecurity "restricted:latest".
Nguyên nhân thường gặp	Values ghi đề thiết lập an toàn mặc định của chart (chạy bằng root, cho leo thang đặc quyền, thiếu seccomp profile, thêm capability).
Chẩn đoán	kubectl -n dev-eks describe rs -l app=<service>-dev; thông báo liệt kê từng vi phạm.
Xử lý	Bỏ phần ghi đề trong file values. Không nối label của namespace; nếu workload thật sự cần, ghi lại ngoại lệ trước.

6.9 IR-09 Lời gọi giữa các service bị timeout

Triệu chứng	Một trang hoặc bước checkout lỗi gRPC DeadlineExceeded / Unavailable sau một thay đổi; pod đích vẫn khỏe.
Nguyên nhân thường gặp	Bên gọi không có trong <code>networkPolicy.allowFrom</code> của bên được gọi, hoặc port đã đổi.
Chẩn đoán	<code>kubectl -n dev-eks get networkpolicy <target>-dev -o yaml</code> ; thử gọi từ một pod mang label của bên gọi so với một pod khác.
Xử lý	Thêm tên release của bên gọi vào <code>allowFrom</code> của bên được gọi (RB-06).

6.10 IR-10 Không vào được shop từ internet

Triệu chứng	Địa chỉ shop bị timeout, hoặc EXTERNAL-IP cứ ở <pending>.
Nguyên nhân thường gặp	Load balancer đang được tạo (DNS có thể mất vài phút); không có pod frontend nào khỏe; Service đã bị xóa.
Chẩn đoán	<code>kubectl -n dev-eks describe svc frontend-external-dev (events)</code> ; <code>kubectl -n dev-eks get pods -l app=frontend-dev</code> ; xem tình trạng instance trong EC2 console, mục Load Balancers.
Xử lý	Chờ tạo xong; sửa frontend (IR-06/IR-07); để Argo CD tạo lại Service (sync frontend-external-dev).

6.11 IR-11 Pod bị Pending hoặc node NotReady

Triệu chứng	Pod nằm ở Pending với <code>Insufficient cpu/memory</code> , hoặc một node ở trạng thái NotReady.
Nguyên nhân thường gặp	Tổng request vượt khả năng của 2 node t3.medium (ví dụ sau khi scale); một node gặp lỗi.
Chẩn đoán	<code>kubectl describe pod <pod></code> (thông báo của scheduler); <code>kubectl describe node <node></code> (resource đã cấp, conditions); <code>kubectl top nodes</code> .
Xử lý	Giảm replica hoặc request, hoặc tăng kích thước node group (RB-09). Node lỗi trong managed node group sẽ được EKS thay; nếu không, cordon, drain rồi terminate instance đó.

6.12 IR-12 HPA hiện metrics <unknown>

Triệu chứng	<code>kubectl -n dev-eks get hpa</code> hiện cpu: <unknown>/70%; frontend không scale.
Nguyên nhân thường gặp	Metrics Server không có hoặc chưa sẵn sàng, hoặc pod không đặt CPU request.
Chẩn đoán	<code>kubectl -n kube-system get deploy metrics-server</code> ; <code>kubectl top pods -n dev-eks</code> ; <code>aws eks describe-addon --cluster-name khaipd18-eks-cluster --addon-name metrics-server</code> .
Xử lý	Apply lại Terraform để add-on metrics-server được cài và chạy ổn định; giữ <code>resources.requests.cpu</code> trong values.

6.13 IR-13 Không drain được node

Triệu chứng	kubectl drain hoặc lần cập nhật node group lặp mãi với Cannot evict pod as it would violate the pod's disruption budget.
Nguyên nhân thường gặp	Drain sẽ khiến frontend không còn pod nào chạy, điều mà PodDisruptionBudget không cho phép. Đây là cơ chế bảo vệ có chủ đích.
Chẩn đoán	kubectl -n dev-eks get pdb; xem pod frontend đang chạy ở đâu và node khác còn chỗ không.
Xử lý	Tạo chỗ trên node khác (tăng node group) để pod bị evict được xếp lại, rồi thử lại. Không xóa PDB.

6.14 IR-14 Giỏ hàng bị trống

Triệu chứng	Người dùng báo giỏ hàng tự nhiên trống.
Nguyên nhân thường gặp	redis-cart khởi động lại; dữ liệu nằm trong volume emptyDir nên mất khi khởi động lại.
Chẩn đoán	kubectl -n dev-eks get pod -l app=redis-cart-dev (tuổi pod, số lần restart).
Xử lý	Đây là hành vi đã biết của bản demo. Cần kho lưu bền vững (ElastiCache hoặc PersistentVolume) trước khi dùng thật.

7 Khôi phục sau thảm họa

Mất gì	Khôi phục thế nào
Toàn bộ cluster hoặc tài khoản	RB-01 dựng lại mọi thứ từ Git: hạ tầng (Terraform), ứng dụng (Argo CD), image (CI). Khoảng 45 phút.
Terraform state	Bucket S3 có versioning: khôi phục phiên bản object trước đó của dev / terraform.tfstate.
Một lần deploy lỗi	RB-05 (revert commit đổi tag).
Dữ liệu giỏ hàng	Không khôi phục được (không lưu bền vững).
Cấu hình Grafana và dữ liệu lịch sử của Prometheus	Không được backup; dashboard có lại cùng chart, dữ liệu lịch sử thì không.

Phụ lục A. Service và port

Service	Ngôn ngữ	Port	Nhận lời gọi từ	CPU req/limit	Memory req/limit
frontend	Go	80 → 8080	any (LoadBalancer)	100m / 200m	64Mi / 256Mi
checkoutservice	Go	5050	frontend	100m / 200m	64Mi / 128Mi
productcatalogservice	Go	3550	frontend, checkout, recommendation	100m / 200m	64Mi / 128Mi
shippingservice	Go	50051	frontend, checkout	100m / 200m	64Mi / 128Mi
cartservice	C#	7070	frontend, checkout	200m / 300m	64Mi / 128Mi
adservice	Java	9555	frontend	200m / 300m	180Mi / 300Mi
currencyservice	Node.js	7000	frontend, checkout	100m / 200m	64Mi / 128Mi
paymentservice	Node.js	50051	checkout	100m / 200m	64Mi / 128Mi
emailservice	Python	8080	checkout	100m / 200m	64Mi / 128Mi
recommendationservice	Python	8080	frontend	100m / 200m	250Mi / 500Mi
redis-cart	Redis	6379	cartservice	100m / 200m	64Mi / 128Mi

frontend-external-dev là Service kiểu LoadBalancer (80 → 8080) trở vào các pod frontend-dev.

Phụ lục B. Lệnh thường dùng

```
# Trạng thái tổng thể
kubectl -n argocd get applications
kubectl -n dev-eks get deploy,pods,svc,hpa,pdb,networkpolicy

# Một service
kubectl -n dev-eks describe pod -l app=<service>-dev
kubectl -n dev-eks logs deploy/<service>-dev --tail 100 [--previous]
kubectl -n dev-eks rollout status deploy/<service>-dev

# CI
gh run list --limit 10
gh run view <run-id> --log-failed
gh workflow run <language>-services-ci.yaml --ref main

# Render một release ở local
helm template <service>-dev helm-charts -f gitops/dev-eks/values-<service>.yaml -n dev-eks

# Terraform (máy local, chỉ khi khẩn cấp)
cd terraform && terraform init && terraform plan
terraform force-unlock <lock-id>
```

Phụ lục C. Phiên bản

Thành phần	Phiên bản	Khai báo ở
Terraform	1.14.8	terraform.yaml, README
AWS provider	6.39.0	terraform/versions.tf
Kubernetes (EKS)	1.35	terraform/variables.tf
VPC CNI add-on	v1.21.1-eksbuild.7	terraform/variables.tf
CoreDNS add-on	v1.13.2-eksbuild.4	terraform/variables.tf
kube-proxy add-on	v1.35.3-eksbuild.2	terraform/variables.tf
Metrics Server add-on	v0.9.0-eksbuild.11	terraform/variables.tf
kube-prometheus-stack	84.4.0	gitops/argocd/monitoring.yaml
Argo CD	stable manifest	README, step 3
Checkov	3.3.22	.github/workflows/*
Trivy	0.75.0	.github/actions/trivy-scan
Redis (cart store)	8.10.2	gitops/dev-eks/values-redis-cart.yaml